Scaling Neural Networks in Soft Actor-Critic

ELE680 - Deep Learning Project Report

Mahdi Nasser Malvin Varpe

November 2026

Abstract

We examined whether larger actor–critic networks improve Soft Actor-Critic (SAC) performance on `HalfCheetah-v5`. We trained eight architectures for one million environment steps each, with three seeds per condition (24 runs). Widening a two-layer network from 256 to 1024 units reduced observed mean final return from 11,906 to 11,084. An unmodified eight-layer network showed large between-seed variation ($8{,}247 \pm 5{,}072$). Tested separately on that deep network, input normalization and residual connections achieved $12{,}951 \pm 481$ and $12{,}674 \pm 1{,}539$, while LayerNorm achieved $8{,}153 \pm 5{,}016$ (mean $\pm$ sample SD across seeds). In this environment and training budget, architectural choices and training reliability mattered more than parameter count alone.

1 Introduction

Larger neural networks can learn richer representations, but adding capacity does not necessarily improve deep reinforcement learning (DRL). In an actor–critic algorithm, the policy changes the data collected during training, while its updates depend on estimates from a learned critic. More parameters can therefore make optimization harder without improving the agent's behavior. Previous work has examined this problem and proposed architectural changes to support deeper networks [2, 6].

We ask how width, depth, and three individual architectural modifications affect Soft Actor-Critic (SAC) when the algorithm, environment and interaction budget remain fixed. We first compare two-layer networks with 256, 512 and 1024 hidden units, then networks with two, four and eight layers at width 256. After seeing considerable variation among the naive eight-layer seeds, we chose that model for one-at-a-time tests of input normalization, residual connections and LayerNorm. This choice was informed by the depth results rather than fixed before training.

Our contribution is an experimental SAC pipeline in which actor and critic architectures can be changed without altering the training protocol, together with separate implementations and tests of the three interventions. We trained 24 million-step runs, checked their recorded outputs and compared both final returns and learning trajectories. We treated architecture as an experimental variable rather than modifying SAC for each run. Network dimensions and the three component switches were controlled through configuration, while evaluation, seed handling and data recording followed a shared protocol. This made the comparisons easier to audit and allowed us to revisit individual learning curves when averages concealed unusual behavior. It did not remove every confound, particularly because the actor and critics changed together, but it kept the experimental question narrow.

The interventions were inspired by SimBa, but we did not implement its combined architecture or reproduce its broader benchmarks [6]. Our aim is to understand which changes helped in our controlled setting, rather than to propose a new reinforcement-learning algorithm.

2 Methods and Materials

2.1 Environment, SAC implementation and design choices

We used the state-based `HalfCheetah-v5` Gymnasium MuJoCo environment (17 state features and six bounded continuous actions), rather than visual observations or a fixed supervised dataset. The agent generated its own replay data through interaction. Choosing one environment allowed us to keep the algorithm and task constant over the full architecture matrix, but limits external validity. Training ran serially on a UiS Gorina4 Tesla P100.

We implemented SAC following Haarnoja et al. [4] and the CleanRL continuous-action reference [3]. In particular, the actor's action-sampling and corrected log-probability calculation adapt the cited CleanRL code, and its update schedule informed our implementation. For this study, we built configurable actor/critic architectures, separate training and evaluation environments, per-run logging and verification. The same SAC training loop was used for all conditions. Both the actor and the online and target critics received each architecture change, so our experiments compare the full actor–critic system, not the effect of changing either network separately.

2.2 Scaling matrix and single-component ablations

The complete matrix comprised 2×256, 2×512, 2×1024, 4×256, 8×256, and three modified versions of 8×256. Here $d \times w$ denotes d hidden fully connected layers of width w, with ReLU activations. The unmodified 2×256 run group was reused as the baseline in both scaling comparisons rather than rerun or counted twice. Increasing width changes the size of each hidden layer; increasing depth adds width-preserving hidden transformations. These are controlled *architectural* comparisons, but are not parameter-count-matched.

The isolated modifications deliberately target different locations in the computation. For **InputNorm**, an online per-feature Welford estimator updated once for each decision-state observation, including during random warm-up. A shared, cached transform normalized observation features for the actor and twin critics; actions, rewards and transitions stored in replay retained their original representations. Evaluation received a frozen snapshot, preventing evaluation episodes from changing the normalization state. This design tests whether managing the *input scale* helps a deep network.

For **Residual**, seven identity shortcuts connected successive width-matched hidden transformations without adding trainable parameters. Normalization was disabled, so a changed optimization path rather than additional learned capacity was the relevant intervention [5]. For **LayerNorm**, independent learnable normalization layers followed each of the eight hidden linear transformations, before ReLU, in the actor and critics [1]. Neither residual connections nor input normalization was enabled for that condition. Its affine scales and biases added 12,288 online parameters, so the control comparison is not strictly parameter matched. These components reflect ideas from SimBa but were applied separately; the experiments do not reproduce its complete combined architecture [6].

2.3 Training controls and evaluation measures

Each configuration used training seeds 0, 1 and 2 and 10^6 environment interactions per seed: 24 distinct scientific training runs. Common settings included replay capacity 10^6, minibatch 256, 5,000 random-action warm-up steps, discount $\gamma = 0.99$, target averaging $\tau = 0.005$, actor learning rate 3×10^{-4} and critic/entropy learning rate 10^{-3}. We retained the compensated policy-update frequency of two, automatic temperature tuning, action squashing, replay treatment and episode/reset behavior. Configurations were resolved and saved for every run, so they could be checked independently of the intended settings.

The policy was evaluated *deterministically* in an environment separate from training, for five episodes every 25,000 steps (40 recorded periodic points), followed by a separate ten-episode final evaluation at one million steps. Within a run, evaluation reset seeds were fixed and reused across checkpoints; this makes policy comparisons at different training stages less sensitive to fresh start-state randomness, but the checkpoints are *not independent replications*. The final ten-episode mean per trained seed is one observation. Tables present the across-training-seed mean and *sample* SD ($n = 3$, divisor $n - 1$), not within-evaluation episode SD, standard error, or a confidence interval.

We also summarize *learning AUC*: for each seed we integrate its 40 periodic mean returns with the trapezoidal rule and divide by the covered step interval (25k–1M). This reports average performance *during* learning rather than just at the final checkpoint. The early 0–25k interval and the separate ten-episode final evaluation are not part of this AUC. No smoothing was applied to the plotted curves; shaded regions represent one across-seed sample SD at each recorded point.

2.4 Quality assurance and provenance

Before the million-step experiments, we tested network outputs, gradient flow, evaluation isolation and checkpoint behavior on CPU and CUDA, including short training runs. Each scientific run recorded its resolved configuration, Git revision, software and GPU environment, evaluation returns and final checkpoint. All 24 runs completed and passed the independent artifact checks. Their original outputs are archived separately; the implementation repository contains a 24-run inventory and scripts to reproduce the published

summaries.[1] Different architecture groups were run at documented scientific source revisions, with complete hashes retained in their metadata. Preflight tests were not counted as scientific trials.

The intended rule was exclusive use of physical GPU 0. For LayerNorm seeds 1 and 2, third-party CUDA processes were observed in 50 of 2,903 and 242 of 2,886 monitor samples, respectively. Both training programs finished and their artifact checks passed, but the external monitoring runners reported these GPU-exclusivity deviations. We retain both results with this qualification; the logs do not establish whether their learned policies were affected.

3 Results

3.1 Final performance across 24 runs

Table 1 reports the final ten-episode evaluation and normalized periodic learning AUC, aggregated across three independently trained seeds. Increasing the two-layer width from 256 to 1024 raised actor-plus-online-critic parameters from 218k to 3.23M yet reduced observed mean final return from 11,906 to 11,084. Four layers at width 256 were similar to the two-layer reference; the naive eight-layer model performed less reliably. Its seed 2 finished at 2,392, while seeds 0 and 1 finished at 11,325 and 11,023, respectively. The full seed scatter in Figure 1 and complete depth mean±SD curves in Figure 3 prevent a best-seed-only interpretation; the original naive eight-layer seed-2 run remains in every reported aggregate.

The learning curves show a similar pattern. At 500k interactions, the periodic mean return was about 10,553 for 2×256 and 8,675 for 2×1024. The wider model also had lower normalized learning AUC ($8{,}987 \pm 1{,}422$ versus $9{,}987 \pm 146$). Four layers at width 256 reached $11{,}884 \pm 1{,}838$ at the final evaluation, close to the baseline mean but with greater variation across seeds. The naive eight-layer model had a final sample SD above 5,000, and its weak seed had already fallen behind early in training. These are outcomes at our fixed training budget; the experiment does not establish how the models would behave with longer training or different hyperparameters.

Table 1: Final return and normalized learning AUC as mean ± *between-training-seed sample SD* ($n = 3$). Parameters count actor and two online critics, not target copies. † marks a GPU-execution deviation in LayerNorm seeds 1 and 2.

Condition	Parameters	Final return	Learning AUC
2×256	217,870	$11{,}906 \pm 497$	$9{,}987 \pm 146$
2×512	828,942	$11{,}645 \pm 482$	$9{,}634 \pm 645$
2×1024	3,230,734	$11{,}084 \pm 1{,}595$	$8{,}987 \pm 1{,}422$
4×256	612,622	$11{,}884 \pm 1{,}838$	$9{,}779 \pm 1{,}422$
Naive 8×256	1,402,126	$8{,}247 \pm 5{,}072$	$7{,}458 \pm 4{,}494$
InputNorm 8×256	1,402,126	$12{,}951 \pm 481$	$10{,}804 \pm 529$
Residual 8×256	1,402,126	$12{,}674 \pm 1{,}539$	$10{,}270 \pm 800$
LayerNorm $8 \times 256^{\dagger}$	1,414,414	$8{,}153 \pm 5{,}016$	$6{,}852 \pm 4{,}048$

3.2 Isolated deep-network interventions

Compared with the naive 8×256 mean of 8,247, **InputNorm** had the highest final return at $12{,}951 \pm 481$, a difference of +4,705 (+57.0%). Its three seeds finished between 12,396 and 13,232. **Residual** reached $12{,}674 \pm 1{,}539$ (+53.7%), with individual seeds between 11,520 and 14,421. Both interventions also increased learning AUC. At 250k interactions, periodic mean returns were about 7,419 for the naive model, 10,189 for InputNorm and 9,174 for Residual, suggesting that the differences emerged during learning rather than only at the final checkpoint.

LayerNorm alone reached $8{,}153 \pm 5{,}016$, close to the naive mean. Its seed 0 remained weak (2,405), while seeds 1 and 2 reached 10,411 and 11,644. The full learning curves in Figure 4 retain all three seeds per condition. In particular, the stronger InputNorm mean is not the result of removing the low naive seed.

Runtime was recorded as a secondary practical cost, not an equally controlled independent variable. Median wall times across the three seeds were about 5.29 h for 2×256, 5.21 h for 2×1024, 6.76 h for 4×256, and 9.79 h for naive 8×256. InputNorm, Residual and LayerNorm 8×256 had median observed times of approximately 9.86, 10.96 and 12.18 h. The nonmonotonic width timings and known shared-server activity show why these

[1] `https://github.com/mahdinass/ele680-scaling-drl`

numbers should not be mistaken for clean isolated throughput benchmarks. A fixed number of interactions controls sample exposure; it does *not* impose a fixed optimization time or GPU occupancy.

3.3 Training diagnostics of the weak deep-network seed

The weak naive 8×256 seed 2 completed normally and passed the same artifact checks as the other runs. In the final 100k training steps, the median entropy-temperature coefficient α was approximately 0.230, 0.202 and 0.034 for seeds 0, 1 and 2; median critic temporal-difference losses were 77.1, 81.0 and 1.9. These differences describe the training dynamics but do not explain the weak policy. A small TD loss does not guarantee an accurate critic, and at 50k steps seeds 0 and 2 had similar α values (0.036 and 0.037) despite their later divergence.

4 Discussion

4.1 What scaling changed

Our clearest finding is that adding layers or hidden units did not reliably improve SAC at the same interaction budget. Two naive 8×256 seeds learned policies with returns near 11,000, while the third finished at 2,392. The deep architecture could therefore learn a reasonably strong policy, but its outcome was less consistent across these three initializations. This distinction matters: the poor mean does not show that an eight-layer network lacks the capacity to solve the task.

Width scaling told a similar story. The 2×1024 model had nearly fifteen times as many actor-plus-online-critic parameters as the 2×256 baseline, yet it had lower mean final return and learning AUC. The four-layer model nearly matched the baseline mean, but its between-seed SD was much larger. We interpret these results as evidence that additional capacity was not the main limitation of our particular SAC setup. This is consistent with the optimization concerns discussed by Bjorck et al. [2], although we did not test the spectral normalization proposed in that work.

4.2 Interpreting the ablations

InputNorm gave the most encouraging results. All three seeds finished above 12,396, its between-seed spread was smaller than that of the naive deep model, and its learning AUC was highest among our conditions. One possible explanation is that placing state features on a more consistent scale makes the deeper actor and critics easier to optimize. Residual connections also improved the observed returns, perhaps because identity shortcuts preserve a direct path for signals and gradients. We did not measure feature conditioning or gradient propagation directly, so these remain hypotheses rather than demonstrated mechanisms.

LayerNorm produced a different outcome. Its strong and weak seeds resembled the variable naive model more than the consistently strong InputNorm runs. Normalizing observations and normalizing hidden activations are not equivalent operations, and our LayerNorm placement and unchanged learning rates may have affected optimization. We cannot determine which factor mattered from the present runs. More broadly, our one-component tests should not be treated as a replication of the combined SimBa architecture [6].

We also need to account for how the ablation target was selected. We chose naive 8×256 after observing its seed variability, and the reported 57% mean improvement with InputNorm is strongly influenced by the control's unusually low seed 2. Comparing medians instead gives approximately 11,023 for naive depth and 13,226 for InputNorm, still a positive descriptive difference but a less dramatic one. With only three seeds, we would not claim a statistically established gain or a reliable variance reduction across future training runs.

The weak deep-network seed's low late-training entropy coefficient and critic loss are worth investigating, but neither identifies the cause. In particular, a successful seed had a similarly small entropy coefficient early in training, and TD loss depends on the learned policy and value targets. The logs suggest a different learning trajectory, not a specific failure mechanism.

The seed-2 trajectory shows why examining individual runs mattered. At 100k interactions it reached only about 1,872 return, whereas the other two naive deep seeds had already reached approximately 6,400 and 6,523. It remained well behind throughout training rather than collapsing near the end. We therefore treated it as a weak but valid training outcome and retained it in the aggregate. Removing it would have hidden much of the observed variability of the naive eight-layer configuration. When evaluating the ablations, we consequently considered learning progress and the consistency of final scores together. InputNorm was encouraging not only because its mean exceeded the naive control, but because all three runs reached strong final returns and

its learning AUC was higher. That is a more useful finding for selecting a follow-up experiment than the 57% mean difference alone.

4.3 Limitations and next steps

Our comparisons cover one MuJoCo task, three seeds per condition and one million interactions per run. We kept the SAC settings fixed rather than retuning them for larger models, and width/depth configurations were not matched by parameter count. Because the actor and critics changed together, we also cannot tell which network contributed most to the outcome. Periodic evaluations used five episodes with fixed reset seeds, while final evaluations used ten episodes; these reduce evaluation noise but do not eliminate uncertainty. Learning AUC and final return describe different aspects of performance, and matching environment steps does not match wall-clock compute.

The architecture groups were trained at documented, separately verified code revisions rather than one identical binary. Two LayerNorm runs also encountered third-party GPU activity despite completing and passing artifact checks. We retained those results, but their execution conditions were not fully exclusive; the monitoring data cannot show whether the policy returns were affected. These qualifications matter when interpreting LayerNorm's variable outcomes.

If we extended the study, we would first repeat InputNorm and Residual with more seeds and a second locomotion environment. We would then inspect critic estimates and gradient propagation to test the proposed mechanisms, rather than infer them from returns alone. For the experiments completed here, our main lesson is that the learning trajectory and consistency across runs told us more than parameter count or a single best score.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL `https://arxiv.org/abs/1607.06450`.

[2] Nils Bjorck, Carla P. Gomes, and Kilian Q. Weinberger. Towards deeper deep reinforcement learning with spectral normalization. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL `https://papers.neurips.cc/paper_files/paper/2021/hash/4588e674d3f0faf985047d4c3f13ed0d-Abstract.html`.

[3] CleanRL contributors. CleanRL: Sac continuous-action reference implementation. `https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/sac_continuous_action.py`, 2026. Implementation reference; action-sampling/log-probability code adapted; consulted October 2026.

[4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 2018. URL `https://proceedings.mlr.press/v80/haarnoja18b.html`.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

[6] Hojoon Lee, Dongyoon Hwang, Donghu Kim, Hyunseung Kim, Jun Jet Tai, Kaushik Subramanian, Peter R. Wurman, Jaegul Choo, Peter Stone, and Takuma Seno. SimBa: Simplicity bias for scaling up parameters in deep reinforcement learning. In *International Conference on Learning Representations*, 2025. URL `https://proceedings.iclr.cc/paper_files/paper/2025/hash/741aab8b41a2987867acc9939ad50383-Abstract-Conference.html`.

Figures

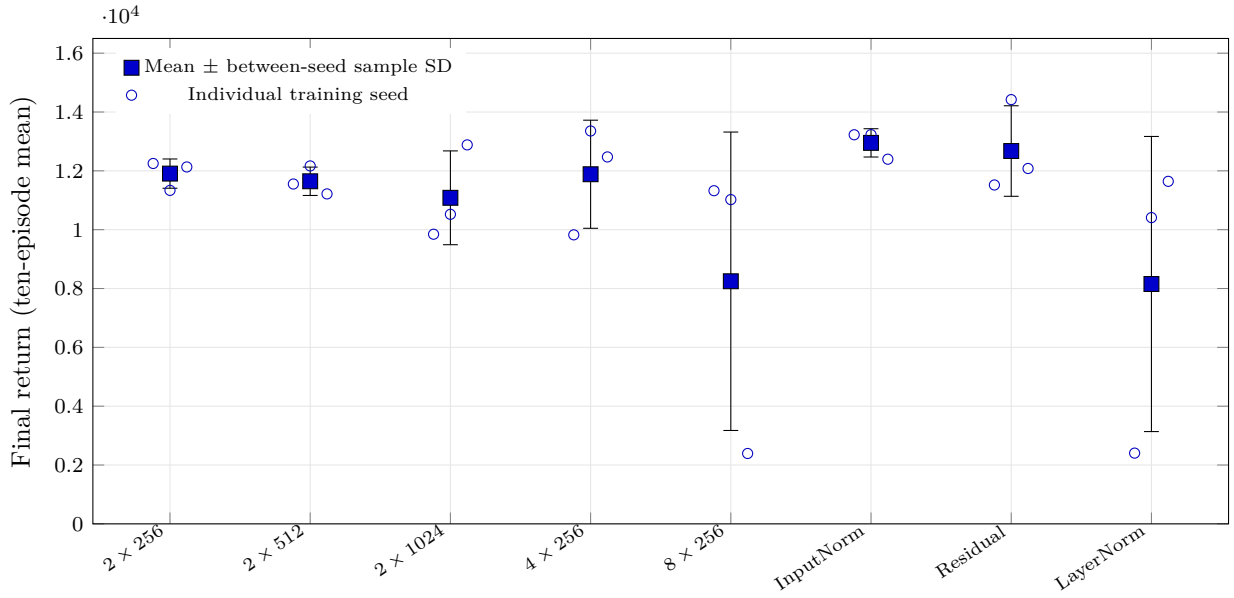


Figure 1: Final ten-episode evaluation return for all 24 runs: individual seed means (circles) and each architecture's mean ± between-seed sample SD (squares and bars; $n = 3$). LayerNorm seeds 1 and 2 had recorded GPU-exclusivity deviations.

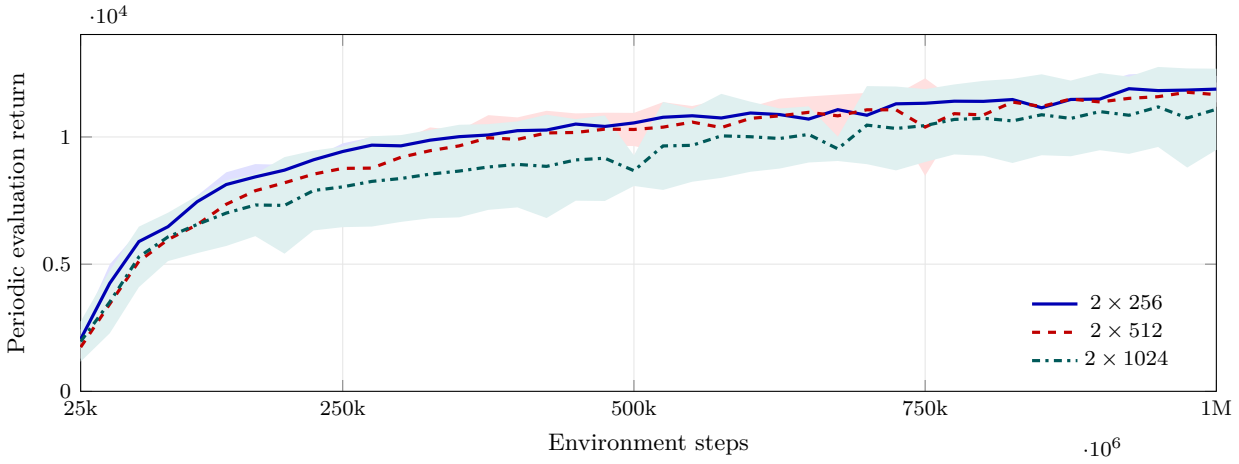


Figure 2: Width-scaling learning curves. Lines show mean deterministic five-episode return over three seeds at each of 40 checkpoints; shaded bands show ±1 between-seed sample SD.

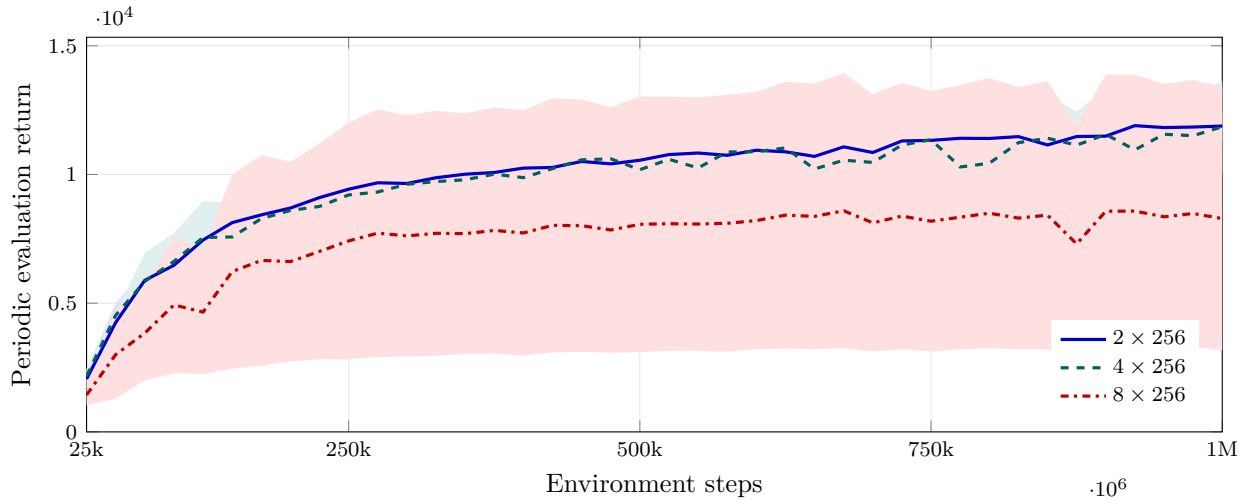


Figure 3: Naive depth-scaling learning curves: mean deterministic five-episode return over three seeds and ±1 between-seed sample SD at 40 checkpoints. The 2×256 baseline is shared with the width comparison.

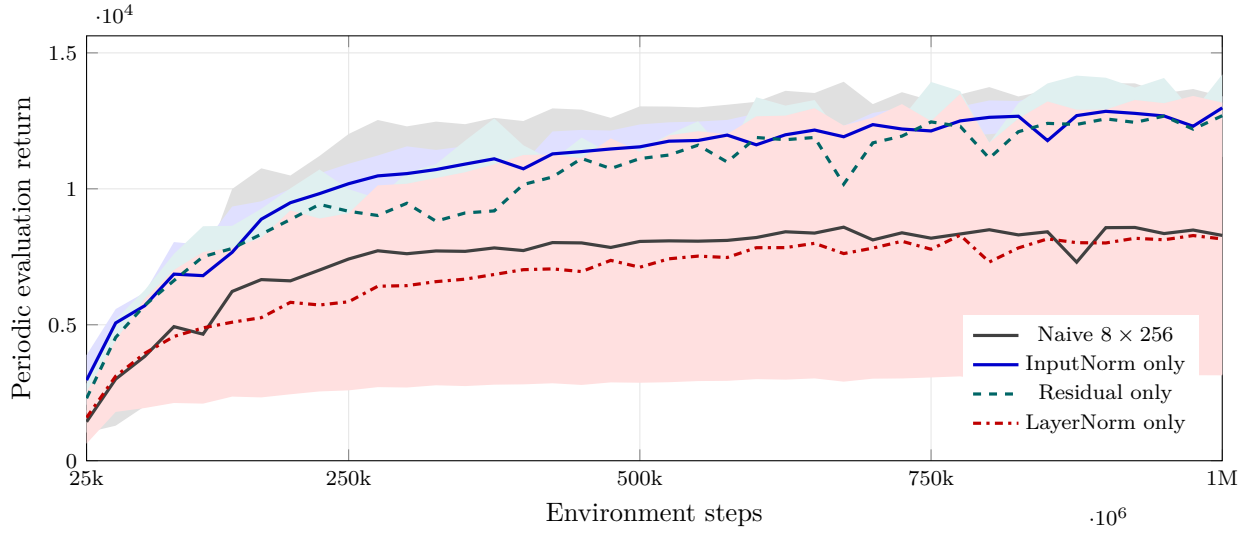


Figure 4: Eight-layer ablations: mean deterministic five-episode return over three seeds and ±1 between-seed sample SD at 40 checkpoints. All seeds are included; LayerNorm seeds 1 and 2 had GPU-exclusivity deviations.